

Black Cat Counter Assignment Report

CS34020 Computer Vision

Alessio Pomo (alp104@aber.ac.uk)

Contents

<u>Introduction</u>	<u>1</u>
1. <u>Workshops</u>	<u>2</u>
2. <u>Cat (Object) Detection</u>	<u>3</u>
3. <u>Colour Detection (Instance Segmentation)</u>	<u>4</u>
4. <u>Black Cat Counting (Object Tracking)</u>	<u>5</u>
5. <u>Optional / Additional Features</u>	<u>6</u>
<u>Self Evaluation</u>	<u>7</u>
<u>References and Statements</u>	<u>7</u>

Introduction

This report documents the development of a computer vision system for counting objects of a certain colour, in this case black cats, in both image and video formats. The program was built using the provided YOLOe .ONNX segmentation model and has the ability to detect cats in each frame, determine each cat's colour using the model's segmentation masks, as well as tracking detected cats across frames in order to avoid duplicate counts, even with abrupt movement and inconsistencies in detection present.

The core pipeline of the program (object detection, colour classification through masking, object tracking / counting) works reliably on the provided test footage, consistently recognising cats and accurately classifying them by their colours while catching the majority of duplicate detections.

The main limitations are in specific cases the brief explicitly flags as difficult, for example a cat that leaves the frame and later reappears, or one that is obscured for an extended period. These can and often are counted again, since a persistent way of recognising and memorising a cat it can no longer see has not been developed.

Detection confidence is also significantly lower on video frames than on still images, most likely due to motion blur, compression, processing power requirements etc. However lowering the confidence threshold and compensating with additional filtering logic has seemed to mitigate this effectively enough.

Beyond just the core requirements stated in the assignment brief, this implementation includes several additional features, including classification of non-black cats into white, ginger, or tabby categories, a file dialogue menu for selecting the input files with support for both image and video formats, a set of playback controls (pause, fast forward, rewind) and visual toggles (bounding boxes, segmentation mask overlay, a keybind help menu) to increase both usability and ease of debugging.

1. Workshops

Workshop 1:

- Load and display an image from a file with **cv.imshow()** and **cv.imread()**
- Prepare an image for deep learning object detection with **cv.dnn.blobFromImage()** - performs resizing, normalisation and reordering. Object detection is a key part of the assignment.
- Crop a rectangle from an image with **img[0:halfheight, 0:cols]** - Useful for separating Regions of Interest (ROIs), masking and thresholding.
- Change the colour space of an image with **cv.cvtColor()** - Allows you to determine the exact colour, hue, saturation, brightness etc. of (parts of) an image.

Workshop 2:

- Segment a specific colour of pixels in (part of) an image by converting to HSV using **cv.cvtColor()**, using **np.array()** for thresholding to identify the pixels of the specified colour, then **cv.bitwise_and()** to mask and separate the pixels of those colours.
- Calculate the percentage of a specific colour of pixels in a (sub)image by using **cv.countNonZero(threshold)** to find the number of pixels of the specific colour in the image, then dividing the total pixels by the result.
- Detect some features on an object that could help with object tracking by using **Harris or KLT corner detection**, to keep track of the corners of the object while it's moving.

Workshop 3: Discuss the suitability of methods used in this workshop for the assignment task:

- The DNN detection method would be more suitable for detecting moving objects as the Haar cascade method is limited to frontal face detection only. There is a class for detecting cats so we can filter it for that detection only as well as providing a confidence score so we can set a threshold for false positives. Post-processing colour analysis would have to be added to detect black cats specifically. This is possible by adding a threshold for detection of the HSV colour space within the bounding boxes.

Workshop 4: Discuss the suitability of methods used in this workshop for the assignment task:

- Background subtraction methods such as KNN and MoG would be able to help detect moving objects against non-moving backgrounds. This can be used to detect black cats by masking and leaving only the cat, then converting to HSV to determine if it is black or not. Although both optical flow methods can also help detect moving objects, they can't detect a specific object as well as background subtraction methods. False positives would be more common. Object tracking methods would help with creating the area (a bounding box) to capture and mask from. Pose tracking would be less effective as it's designed more for tracking specific points rather than an area such as human joints. Using object tracking to create an area around an object to capture from in an image or video then using background subtraction to separate the object for color analysis would be the most viable solution.

Workshop 5: Discuss how Visual-SLAM could be combined with object detection and tracking to allow an agent (e.g. a robot) to locate the 3D position of the object in relation to the agent:

- Visual SLAM allows a robot to know where it is and create a 3D map around itself. Combining it with the ability to detect objects allows for it to determine how far the object is, in what direction it's moving, whether to avoid or follow it, and its location on a 3D map.

2. Cat (Object) Detection

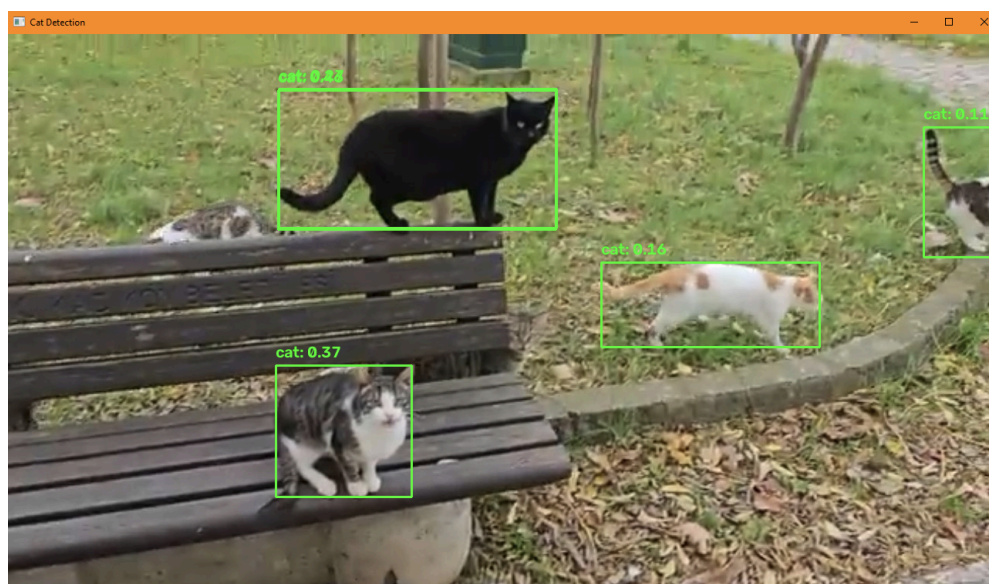
The objective at this stage of the assignment was only to identify cats (using the provided pre-trained model) from a static image. I built upon the structure provided in Workshop 3 (load model, image preprocessing / blob, forward pass, detection loop, bounding box) and adjusted it to be compatible with the new model. These changes included using the `readNetFromONNX` function as the new model was a .onnx file instead of a Caffe model, changing the expected input “blob” size from 300x300 to 640x640 and modifying what was looped over to match the new output format.

Although the assignment brief stated that the normalisation values from the workshop would be suitable, it was not the case in my experience. I suspect this is because the MobileNet-SSD model from the workshop expects input values in the range $[-1, 1]$, while this ONNX export expects $[0, 1]$. I therefore altered the mean and scale values from 127.5 and 0.007843 to (0, 0, 0) and 1/255.0 respectively.

With these changes I was able to get a detection, although I had to lower the confidence threshold as the default value from the workshop appeared to be too high for this specific model. However, confidence remained very low and many cats still went undetected.

After further troubleshooting and research, I realised that the `blobFromImage` function was distorting the image's aspect ratio by squeezing it directly into 640x640 rather than preserving its proportions, resulting in a stretched image. The fix for this issue was to write a “letterbox” function that pads the image to a square before resizing and to use the resulting scale factor instead of the original image width/height to map bounding boxes back to the original image size.

Printing `blob.min()`, `blob.max()`, and `blob.mean()` was essential for diagnosing the normalisation issue and printing the maximum confidence value across detections was similarly useful for the earlier confidence problem.



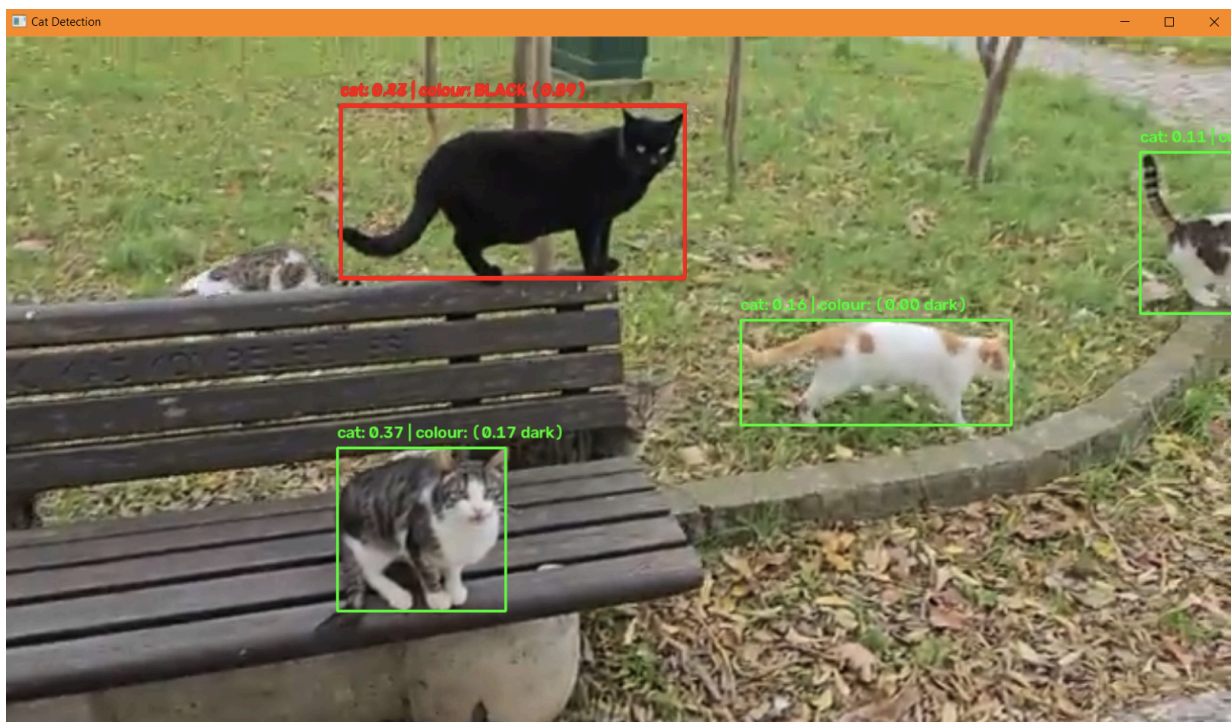
With both fixes in place, detection became accurate and consistent across multiple cats in a single frame as shown here. A remaining issue though was duplicate detections on the same cat. I chose to address this later, once video functionality (and Non-Maximum Suppression) would be implemented.

3. Colour Detection (Instance Segmentation)

The next step was to determine whether each detected cat's colour was black or otherwise. This required creating a segmentation mask from the model's output and applying it to the frame. The brightness (represented by the V channel of HSV, a more reliable measure of darkness than just RGB) of the cat's masked pixels is measured and if the fraction of dark pixels exceeds a set threshold, then the cat is classified to be black.

The mask is calculated using the given formula
$$M(x, y) = \sum_{i=0}^{31} w_i P_i(x, y)$$
 where $M(x, y)$ is the mask, w_i is the weight for prototype image i , and $P_i(x, y)$ is the i th prototype image. Essentially for every pixel, you take each of the 32 prototype masks, multiply by their corresponding weights and add all the results up.

However, instead of using a for loop to iterate over one prototype at a time, NumPy operations (specifically matrix multiplication)[1] can make this calculation much more efficient. In this case, we can compute every pixel simultaneously by reshaping the 32 prototypes (output1) from (32, 160, 160) to (32, 25600), treating each 160x160 mask as a single row of 25600 values instead, then multiplying against the 32 weights in only one operation. The result is reshaped back to (160, 160) to get the final mask.



Testing on a black cat in the image above confirmed that the detection and labelling was correct, with the bounding box drawn in red and the fraction of dark pixels displayed above it, along with the labelling used for other cats.

At this stage, non-black cats were labelled generically, with colour classification for white, ginger, and tabby cats being implemented later as an additional feature. The duplicate detection issue seen here and in the previous section was resolved once support for video formats (and NMS and IoU) was introduced.

4. Black Cat Counting (Object Tracking)

This final part required the combination of the previous parts 2 and 3 with some extra logic needed to work with video files and not just images. The detection, masking, and colour classification logic was wrapped into a single function so it could run once per frame inside the main video loop. Aside from this, some of the key additions were IoU (Intersection over Union) checks and NMS (Non Maximum Suppression)[2].

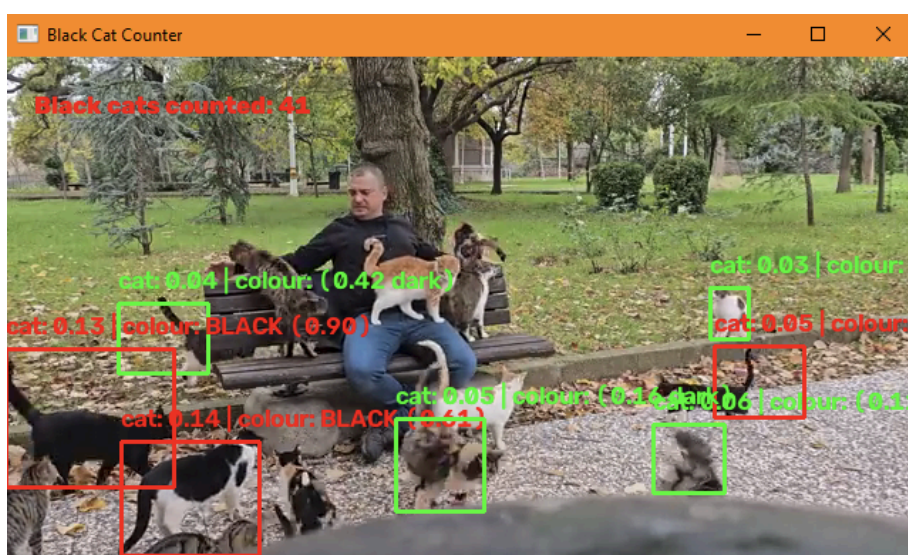
Intersection over Union checks compare each new detection's box against previously tracked cats, using a set overlap threshold to decide whether a detection is the same cat seen before or a new one. This works well in general, but if the detector fails to find a cat for a period of a few frames, it risks being counted again when redetected. To mitigate this, a function was built in to allow tracked detections to persist for a number of frames without being seen before they are discarded, along with giving cats that have already been counted a much longer allowance, since losing and redetecting a cat that has already been counted is a lot more damaging to the quality of results in comparison to losing an unconfirmed detection.

Non Maximum Suppression was used in conjunction with IoU. While IoU works across frames, NMS addresses a different but similar problem, removing duplicate, overlapping boxes within a single frame. NMS is more effective on images than video files but it still provides a decent amount of value especially considering that it works on top of IoU. Like IoU, it does come with its own caveats. If the threshold is set too low, then multiple detections risk merging together into one, while if set too high then it won't have much of an effect at all. I additionally implemented a containment check, which removes any small box mostly contained within a larger one. This was to catch a specific type of duplicate that neither IoU nor NMS consistently addressed for a reason I could not figure out. With this additional check the system had become very robust to duplicate and overlapping boxes.

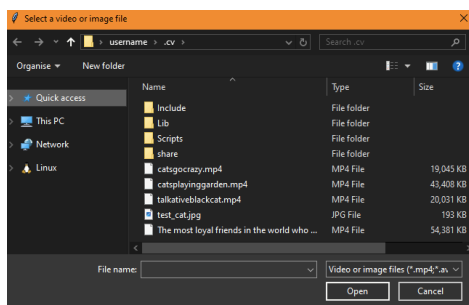
All the thresholds (detection confidence, mask threshold, black-fraction threshold, IoU match threshold, NMS threshold, containment threshold etc.) were assigned to named constants at the start of the script for easier tuning and tweaking. Additional constants were also assigned, such as one to control how often a detection is run per a number of frames (higher values trading accuracy for speed), how many consecutive frames a detection must be active before being confirmed and

counted (reducing false positives from transient noise), and how long a confirmed detection can go unseen before being discarded (reducing recounts but at the cost of memory and visual clutter).

A screenshot of the program at this stage midway through a video, with the total black cat counter in the top left.



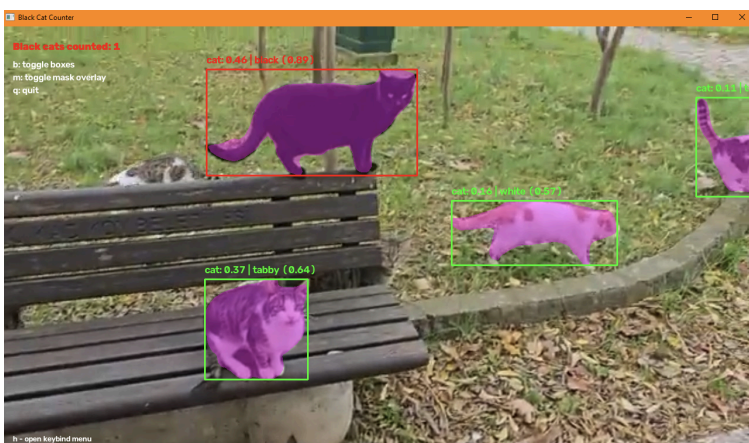
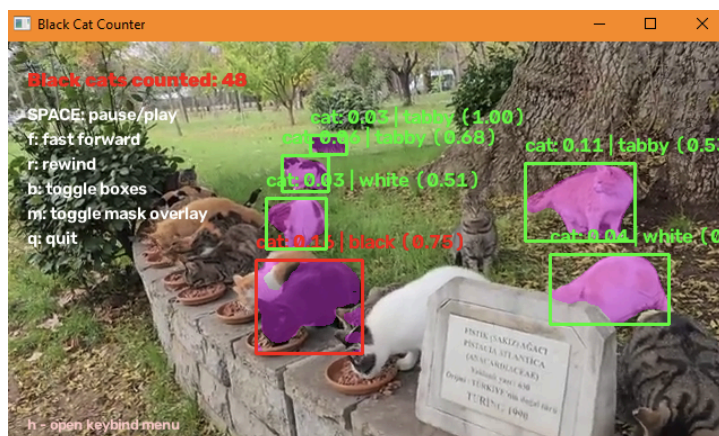
5. Optional / Additional Features



The first additional feature implemented was a file dialog, where running the program would open up a file explorer window allowing you to navigate to and select your desired file, replacing the need to hardcode a video path directly in the script. This was a very quick addition (importing `filedialog` from `tkinter`[3] and configuring the open file window) that drastically improved the usability of the program.

The second feature to be added was colour classification for non-black cats. I used the three colours suggested in the brief (white, ginger, and tabby) and calculated a confidence score for each classification, and displayed it on the bounding box label. The colours are determined by the fraction of the cat's masked pixels whose hue and saturation values fall within the expected range for white or ginger. As tabby is not really a colour, a cat is labelled tabby whenever it has already been determined not to be black and both the white and ginger fractions are low.

Another addition was the implementation of playback keybinds and visual toggles, both to aid debugging and as general quality of life improvements. The playback controls featured include pause/play, fast forward, and rewind, with the number of frames skipped per keypress configurable by adjusting the value assigned to the corresponding constants. Rewinding also removes any black cat counts that were confirmed after the point rewinded to, so the counter accurately reflects only what has actually been seen so far. Playback speed keybinds were attempted but ultimately removed, since the speed of playback was bottlenecked elsewhere, meaning the keybinds could only artificially slow playback down, not speed it up. One of the visual toggles allow the bounding boxes and labels to be hidden to declutter and clean up the display while the other shows the calculated segmentation mask as a semi-transparent overlay. This proved to be particularly useful when debugging and tuning the mask threshold. All keybinds can be shown



or hidden via a help overlay, removing the need to remember each keybind with a persistent but discreet on-screen tooltip of the key to open it.

Finally, the last feature I added was compatibility with both image and video files and the ability to process them correctly and automatically. Images when opened would not have the playback keybinds available but visual toggles would still work as normal.

Self Evaluation

Part 1 - Workshops:

I believe that I've answered all the workshop questions correctly, to a high standard and provided strong links to how the learned content would transfer to the assignment. 60-70%

Part 2 - Cat (Object) Detection:

The object detection is very accurate and consistent across multiple objects per frame and the debugging processes and patches are clearly evidenced. I believe that excellent use of OpenCV to detect cats in static images / single frames has been demonstrated. 70%

Part 3 - Colour Detection (Instance Segmentation):

The mask calculation follows the given formula correctly and is implemented efficiently using vectorised NumPy operations, with a comprehensive explanation to go with it. The screen captures of the mask overlays provided are clearly accurate so I believe that excellent use of OpenCV to segment cats in static images / single frames has been demonstrated. 70%

Part 4 - Black Cat Counting (Object Tracking):

The object tracking has been proven on different types of footage to be reliable and robust. A combination of IoU matching and NMS with a containment check, along with well-tuned thresholds are both explained and implemented effectively, therefore I believe that excellent use of OpenCV to track cats in videos / prevent repeated detections has been demonstrated. 70%

Part 5 - Optional / Additional Features:

Several additional features which greatly increase ease of use as well as help debugging have been developed and successfully implemented including a file dialog menu, playback controls, visual toggles, a help menu and both image and video support. 70%

Part 6 - Report:

Report writing has admittedly been one of the areas I tend to struggle in, however I do think that this report was written to a high standard and to the best of my abilities, I can't think of many improvements that could be made to it at the moment. 50-60%

References and Statements

- [1] <https://numpy.org/doc/2.1/reference/generated/numpy.matmul.html>
- [2] <https://learnopencv.com/non-maximum-suppression-theory-and-implementation-in-pytorch/>
- [3] <https://docs.python.org/3/library/dialog.html#module-tkinter.filedialog>

This report is completely self-written with no AI usage involved. All images and figures were taken from my own personal computer.

The submitted code is self-written with guidance and information learned from the resources listed above in references, as well as the module workshops. No AI usage involved.